

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC MÁY TÍNH VÀ KỸ THUẬT MÁY TÍNH



BÀI TẬP LỚN 1
MẠNG MÁY TÍNH (CO3093)

DEVELOP A NETWORK APPLICATION

Học kì 242

Giảng viên hướng dẫn: Nguyễn Lê Duy Lai

STT	Họ và tên	MSSV	Lớp	Ghi chú
1	Hà Thiên Hải	2210882	TN01	
2	Lê Sĩ Hoàng	2211082	TN01	

Thành phố Hồ Chí Minh, tháng 4 năm 2025

Mục lục

Danh sách hình vẽ	3
Danh sách bảng	3
1 Giới thiệu chung	3
2 Kiến trúc ứng dụng	4
3 Chức năng chính	5
3.1 Tracker	5
3.2 Peer	5
4 Đặc tả ứng dụng	7
4.1 Bencoding	7
4.2 Metainfo file(.torrent file)	7
4.3 Pieces	8
5 Giao thức giao tiếp	9
5.1 Tracker HTTP protocol	9
5.1.1 Tracker GET Request (Announce)	9
5.1.2 Tracker POST request	9
5.1.3 Tracker response	10
5.1.4 Ví dụ phản hồi JSON:	10
5.2 P2P TCP protocol	10
5.2.1 Handshake	10
5.2.2 Message Types	11
6 Thiết kế hệ thống	12
6.1 Class Diagram	12
6.2 Sequence Diagram	13
6.2.1 Active and ShutDown Client	13
6.2.2 File Uploading	14
6.2.3 File Downloading	15
7 Implementation and Testing	16
7.1 Implementation	16
7.2 Testing	16
8 Future Work	18
9 Tài liệu tham khảo	19

Danh sách hình vẽ

1	Kiến trúc BitTorrent bao gồm Tracker trung tâm và các file .torrent .	4
2	Class Diagram của hệ thống	12
3	Sequence Diagram cho quá trình khởi động và tắt Client	13
4	Sequence Diagram cho quá trình tải lên Client	14
5	Sequence Diagram cho quá trình tải xuống Client	15
6	Khởi động Tracker	16
7	Khởi động Client	16
8	Bắt đầu quá trình trao đổi file	17
9	Hoàn tất quá trình trao đổi file	17

Danh sách bảng

1 Giới thiệu chung

Trong kỉ nguyên thời đại số, nhu cầu cho những giải pháp chia sẻ tệp tin một cách hiệu quả và dễ dàng mở rộng ngày càng gia tăng, kể cả đối với nhu cầu cá nhân và thương mại hóa. Đối với phương thức trao đổi tệp tin truyền thống, như kiến trúc client-server, thường bị giới hạn bởi độ lớn băng thông của server trung tâm. Điều đó có thể dẫn đến tắc nghẽn khi phải phục vụ một khối lượng lớn client cùng một lúc. Hơn thế nữa, việc chỉ có một server trung tâm, nếu xảy ra một lỗi gì đó có thể dẫn đến sự sụp đổ của cả hệ thống.

Từ những vấn đề ở trên, phương thức chia sẻ tệp tin Peer-to-Peer (P2P) ra đời để giải quyết các vấn đề trên. Trong hệ thống P2P, tệp tin được chia sẻ trực tiếp giữa những người dùng, giúp hạn chế sự phụ thuộc vào server trung tâm và phân bổ nó đến với các peers. Bằng cách chia nhỏ tệp tin lớn thành những mảnh nhỏ (pieces) và tải chúng từ nhiều peers cùng một lúc, hệ thống P2P có thể gia tăng một cách đáng kể tốc độ tải xuống.

Mục tiêu của dự án này là phát triển một ứng dụng chia sẻ file lai giống BitTorrent, tận dụng được cả ưu điểm của phương pháp tập trung và phân tán. Một máy chủ tập trung được sử dụng để theo dõi các peer và các tệp tin mà họ đang có, trong khi vẫn áp dụng mô hình P2P để cho phép các client trực tiếp tải lên và tải xuống dữ liệu với nhau.

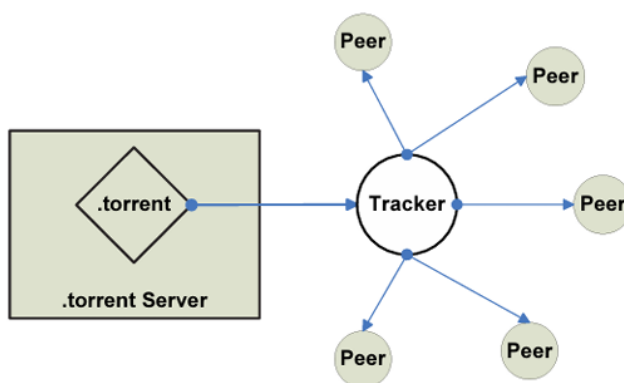
2 Kiến trúc ứng dụng

Trong kiến trúc của chúng tôi, tracker sẽ liên tục cập nhật và duy trì danh sách của tất cả các peer hiện đang seeding (hoặc sẵn sàng để bắt đầu seeding) tập tin. Thêm vào đó, tracker cũng lưu meta-info (ví dụ .torrent file) trong bộ nhớ để phục vụ cho các peers đang cần tìm kiếm tập tin đó. Nó cho phép các peers lấy được những thông tin cần thiết để tải xuống tập tin mong muốn trong swarm

Khi một client muốn seeding một tập, nó có thể quyết định công khai file meta-info tương ứng hay không. Nếu client chia sẻ file meta-info, nó sẽ gửi file .torrent đó lên tracker để lưu trữ. Bất kỳ client nào cũng có thể yêu cầu server cung cấp danh sách tất cả các file meta-info công khai và tải về file .torrent tương ứng với tập mà mình muốn download. File .torrent này sau đó sẽ chứa toàn bộ thông tin cần thiết để client có thể tải tập về.

Theo giao thức của tracker, một client (tức là một peer) ở phía seeding sẽ thông báo với tracker những tập mà nó muốn upload/seeding cho các peer khác, nhưng không gửi dữ liệu thực tế của các tập đó lên server. Khi một client muốn download một tập, nó sẽ yêu cầu tracker danh sách các peer đang seeding tập đó.

Mô hình P2P là yếu tố cốt lõi trong ứng dụng chia sẻ file của chúng tôi. Khi client nhận được danh sách peer đang hoạt động từ tracker, nó sẽ thiết lập nhiều kết nối P2P với các peer đó và bắt đầu tải (leech) các phần (piece) của tập. Trong triển khai của chúng tôi, mỗi peer có thể vừa upload nhiều tập khác nhau, vừa download đồng thời nhiều tập từ các peer khác nhau.



Hình 1: Kiến trúc BitTorrent bao gồm Tracker trung tâm và các file .torrent

3 Chức năng chính

3.1 Tracker

Các chức năng chính của tracker trong hệ thống bao gồm:

- **Quản lý danh sách peer:** Tracker sẽ duy trì danh sách cập nhật của tất cả các peers sẵn sàng seeding cho một tệp tin chia sẻ. Danh sách này sẽ cập nhật linh động bất cứ khi nào tracker nhận được một thông báo *join* (peer bắt đầu seeding) hoặc *leave* (peer dừng seeding).
- **Truy xuất danh sách peers:** Khi một client muốn tải một tệp tin, nó sẽ gửi đến tracker một yêu cầu để nhận danh sách các peer đang seeding tệp tin đó. Tracker sẽ phản hồi với địa chỉ IP và cổng của tất cả các peer đang sẵn sàng.

3.2 Peer

Các chức năng chính của peers trong hệ thống bao gồm:

- **Tải lên đến multi-peer:** Một peer có thể đăng tải đến nhiều peer cùng lúc. Khi một peer hoàn thành việc tải xuống, nó có thể chia sẻ các pieces đến cho các peers khác đang cần tải nó.
- **Tải xuống từ multi-peer:** Một peer có thể tải xuống các tệp bằng cách kết nối với các peer khác. Peer sẽ yêu cầu các mảnh tệp từ các peer dựa trên danh sách do tracker cung cấp.
- **Kết nối đồng thời:** Các peer có khả năng thiết lập nhiều kết nối cùng lúc với các peer khác, giúp client vừa tải xuống các mảnh của nhiều tệp từ nhiều nguồn khác nhau, vừa chia sẻ các mảnh đã có cho những peer khác. Cách tiếp cận song song này giúp tối ưu tốc độ truyền dữ liệu và nâng cao hiệu quả toàn hệ thống.
- **Quản lý mảnh dữ liệu (Piece Management):** Quá trình tải xuống được thực hiện theo từng mảnh nhỏ. Peer sẽ quản lý trạng thái của từng mảnh - như đã tải xong, đang chờ, hay chưa tải - đồng thời đảm bảo không có sự trùng lặp hoặc thiếu sót trong quá trình yêu cầu dữ liệu.

- **Xác thực mảnh dữ liệu (Piece Validation):** Sau khi một mảnh được tải về, peer sẽ kiểm tra tính toàn vẹn của nó bằng cách so sánh giá trị băm (hash) với giá trị trong tệp meta-info (.torrent). Việc này giúp đảm bảo dữ liệu không bị lỗi hoặc hư hỏng trong quá trình truyền.
- **Theo dõi trạng thái tải lên và tải xuống:** Mỗi peer chủ động ghi nhận và cập nhật tiến trình của các tệp đang được tải lên và tải xuống để quản lý hiệu quả việc chia sẻ dữ liệu.
- **Tiếp tục seeding sau khi tải xong:** Khi một peer đã hoàn tất việc tải xuống, nó có thể chuyển sang chế độ seeding - chia sẻ lại dữ liệu với các peer khác. Việc này giúp duy trì tính sẵn sàng và đảm bảo các tệp luôn có thể được truy cập trong mạng chia sẻ.

4 Đặc tả ứng dụng

Ứng dụng được thiết kế dựa trên các đặc điểm của BitTorrent. Ở phần trình bày này chúng ta sẽ tìm hiểu làm thế nào mà hệ thống đáp ứng và vận hành dựa theo giao thức của BitTorrent nhằm hỗ trợ phân phối tệp một cách phi tập trung và hiệu quả.

4.1 Bencoding

Bencoding là một định dạng dữ liệu được sử dụng trong giao thức BitTorrent dùng để mã hóa dữ liệu thành byte strings, integers, list, and dictionaries

- **Byte Strings** được mã hóa như sau:

<Độ dài chuỗi ở hệ mười ASCII>:<dữ liệu chuỗi>

Ví dụ: 5:hello → "hello"

- **Integer** được mã hóa như sau:

i<Số nguyên ở hệ mười ASCII>e

Ví dụ: i2e → 2; i-5e → -5

- **Lists** được mã hóa như sau:

l<bencoded values>e Ví dụ: l5:hello5:worlde → ["hello", "world"]

- **Dictionaries** được mã hóa như sau:

d<bencoded string><bencoded value>e

Ví dụ: d5:hello5:worlde → "hello":"world"

4.2 Metainfo file(.torrent file)

Metainfo file (.torrent file) bao gồm metadata của tệp tin được chia sẻ cũng như là vị trí các peers đang có ở trên mạng chia sẻ torrent (swarm). Trong khuôn khổ của bài tập lớn, để đơn giản hóa hệ thống, chúng tôi chỉ sử dụng những phần quan trọng của kiến trúc tệp tin metainfo. Nội dung của tệp tin metainfo là một bencoded dictionary, bao gồm các keys được liệt kê dưới đây:

- **announce:** URL của tracker.

- **info:** Đây là một từ điển mô tả tệp (hoặc các tệp) trong torrent.
- **piece length:** Số byte trong mỗi piece (integer).
- **pieces:** Chuỗi kết hợp của tất cả các giá trị băm SHA-1 dài 20 byte, mỗi mảnh một giá trị (kiểu `byte string`). Khi tải xuống, bạn có thể kiểm tra tính hợp lệ của mỗi mảnh bằng cách so khớp SHA-1.
- **name:** Tên của tập tin.
- **length:** Kích thước của tập tin ở đơn vị bytes (integer).
- **path:** Vị trí của tập tin đó.

4.3 Pieces

Giao thức BitTorrent chia mỗi tệp thành các “mảnh” (piece) có kích thước bằng nhau để tối ưu việc tải lên và tải xuống giữa nhiều peer. Kích thước mỗi mảnh được quy định trong trường `piece length` của tệp meta-info.

Mỗi mảnh được nhận diện bằng một giá trị băm SHA-1 dài 20 byte, và toàn bộ các giá trị băm này được ghép nối lại thành chuỗi duy nhất trong trường `pieces` của tệp meta-info. Khi một peer yêu cầu tải mảnh, nó sẽ so khớp mảnh nhận được với giá trị SHA-1 tương ứng để đảm bảo mảnh đó không bị thay đổi hoặc hỏng. Nhờ vậy, giao thức luôn duy trì tính toàn vẹn của dữ liệu trong quá trình trao đổi.

5 Giao thức giao tiếp

5.1 Tracker HTTP protocol

Tracker hoạt động như một dịch vụ HTTP, xử lý các yêu cầu được gửi từ client thông qua các phương thức HTTP như GET và POST. Các client gửi yêu cầu đến tracker để thông báo trạng thái seeding của chúng và đồng thời yêu cầu thông tin về các peer nhằm phục vụ cho việc tải xuống.

5.1.1 Tracker GET Request (Announce)

URL gốc (base URL) của tracker là “announce URL” được chỉ định trong tệp metainfo. Ứng dụng sẽ nối thêm các tham số sau vào URL để cung cấp thông tin trạng thái peer và yêu cầu danh sách peer:

- **torrent_id**: Giá trị băm SHA-1 (20 byte) của phần “info” trong tệp metainfo. Tham số này định danh duy nhất nội dung của tệp hoặc thư mục.
- **peer_ip**: Địa chỉ ip mà peer đang lắng nghe kết nối đến
- **port**: Số cổng mà peer đang lắng nghe các kết nối đến.

Ví dụ: http://tracker.example.com/announce?torrent_id=<torrent_id>&peer_id=127.0.0.1&port=6881

5.1.2 Tracker POST request

Như đã đề cập trước đó, tracker của chúng tôi cũng có khả năng lưu trữ các tệp meta-info để chia sẻ thông tin torrent. Chúng tôi giới thiệu yêu cầu này để client có thể gửi tệp metainfo đến tracker nhằm mục đích chia sẻ. Cấu trúc của yêu cầu POST tương tự như yêu cầu GET, với các tham số như peer_id, peer_ip, và port. Tuy nhiên, điểm khác biệt chính là việc thêm vào tham số file, chứa nội dung thực tế của tệp metainfo (tệp .torrent) mà tracker sẽ lưu trữ.

5.1.3 Tracker response

Tracker phản hồi các yêu cầu của client bằng một tài liệu JSON với cấu trúc sau:

- **peers**: Mảng các tuple:
 - **ip**: Phần tử đầu tiên của tuple.
 - **port**: Phần tử thứ hai của tuple.

5.1.4 Ví dụ phản hồi JSON:

```
{  
  "peers": [  
    ("127.0.0.1",6000),  
    ("127.0.0.1",6001)  
  ]  
}
```

5.2 P2P TCP protocol

Giao thức P2P là một phần quan trọng trong hệ thống chia sẻ tập tin, cho phép các máy khách giao tiếp trực tiếp và trao đổi dữ liệu với nhau mà không cần dựa vào máy chủ trung tâm.

Quá trình kết nối giữa các peer được khởi tạo đầu tiên thông qua bắt tay (handshake). Sau đó, các peer bắt đầu yêu cầu và nhận các phần dữ liệu thông qua các loại thông điệp khác nhau. Tất cả các loại thông điệp này (trừ thông điệp bắt tay) đều có một số nguyên 4 byte đặt trước để xác định độ dài của thông điệp sắp nhận. Các loại thông điệp sẽ được mô tả chi tiết bên dưới.

5.2.1 Handshake

Handshake là bước thiết lập kết nối ban đầu giữa hai peer trong hệ thống chia sẻ ngang hàng, đảm bảo cả hai tuân thủ cùng giao thức. Ở đây, để đơn giản hóa quá trình, peer khởi tạo (client) sẽ gửi thông điệp PING (4 byte) đến peer đích (server). Peer nhận kiểm tra PING, nếu hợp lệ sẽ phản hồi PONG (4 byte) để xác nhận.

Client chờ "PONG" trong khoảng thời gian định trước (timeout). Nếu thành công, hai peer bắt đầu trao đổi dữ liệu. Ngược lại nếu thất bại (sai thông điệp, timeout, hoặc lỗi kết nối), quá trình bị hủy để tránh lãng phí tài nguyên. Handshake đảm bảo tính tương thích, loại bỏ kết nối không hợp lệ và thiết lập kênh truyền thông an toàn trước khi truyền dữ liệu chính thức.

5.2.2 Message Types

Sau handshake thành công, các peer trao đổi dữ liệu qua các thông điệp có cấu trúc như sau:

Định dạng chung:

- 4 byte đầu: Độ dài toàn bộ thông điệp.
- Header (JSON): Mô tả loại thông điệp và metadata.
- Data (tùy loại): Dữ liệu nhị phân (nếu có).

REQUEST_CHUNK:

- **Mục đích:** Yêu cầu một phần dữ liệu.
- **Header:** {"command": "REQUEST_CHUNK", "file_name": "...", "chunk_index": N}.

CHUNK_DATA:

- **Mục đích:** Phản hồi dữ liệu hoặc báo lỗi.
- **Header:** {"command": "CHUNK_DATA", "status": "OK/ERROR", "data_length": N}.
- **Data:** Chuỗi byte của chunk (nếu thành công).

ACK/ERR:

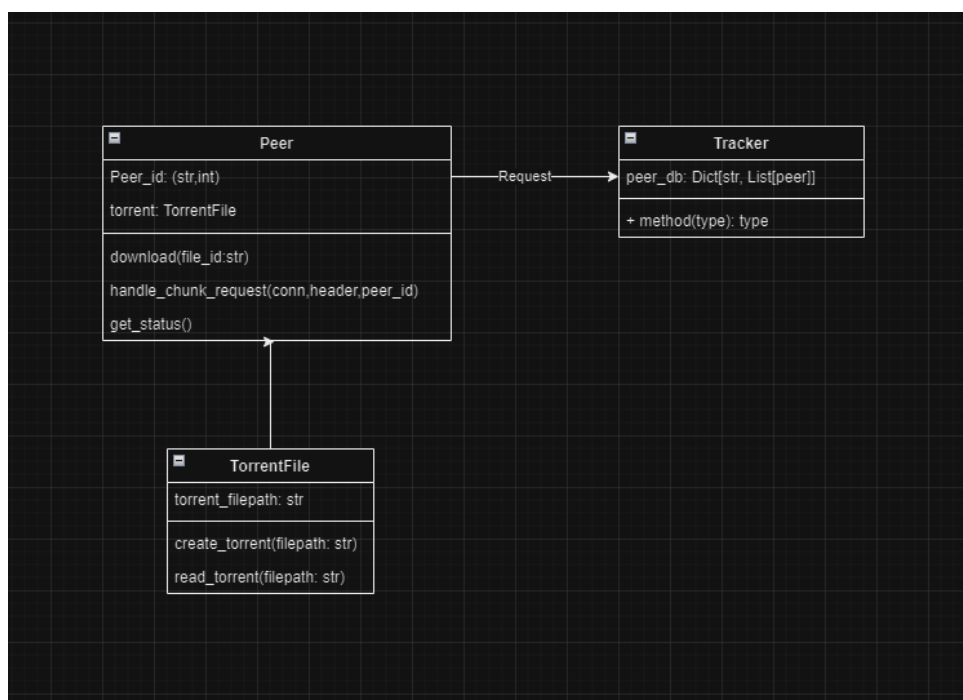
- 3 byte phản hồi: "ACK" (thành công) hoặc "ERR" (lỗi).

6 Thiết kế hệ thống

6.1 Class Diagram

Class diagram (hình 2) mô tả những chức năng và mối quan hệ của các đối tượng chính trong hệ thống

- Lớp Tracker quản lý thông tin của peer. Nó lưu trữ danh sách các peer đang hoạt động ứng với mỗi tập tin và sẽ gửi danh sách đó đến các peer nếu có yêu cầu.
- Lớp Peer sẽ phụ trách việc tải lên (Uploading) và tải xuống (Downloading). Nó sẽ tương tác với Tracker để "announce" nếu muốn seeding hoặc nhận những thông tin cần thiết cho việc tải xuống tập tin.
- Lớp TorrentParse sẽ được dùng để trích xuất metainfo từ file .torrent.



Hình 2: Class Diagram của hệ thống

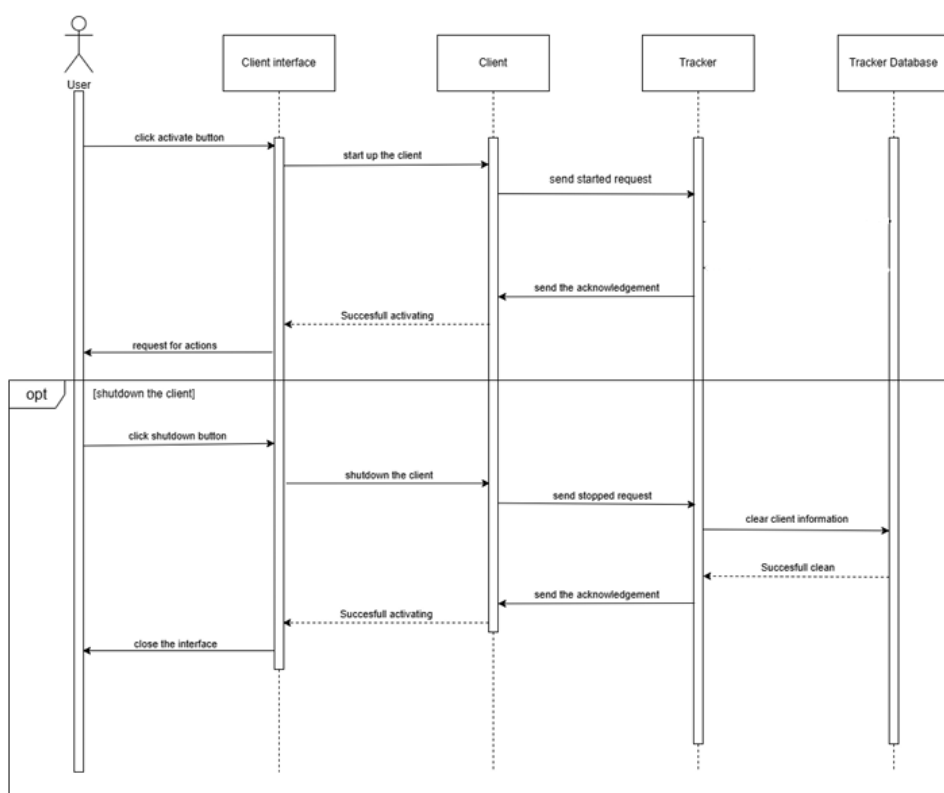
6.2 Sequence Diagram

6.2.1 Active and ShutDown Client

Sequence Diagram (Hình 4) mô tả quá trình kích hoạt và tắt một client với sự tương tác giữa nhiều thành phần: Người dùng, Client CLI, Client, Tracker, và Tracker_DB.

Trình tự kích hoạt bắt đầu khi người dùng nhập gõ lệnh "start" trên terminal. Hành động này kích hoạt Client CLI để khởi động Client. Ứng dụng khách sau đó xác nhận việc kích hoạt thành công với Client Cli, và Cli tiếp tục yêu cầu các hành động tiếp theo từ người dùng.

Quy trình tắt máy tùy chọn được trình bày trong một khối riêng biệt. Nếu Người dùng quyết định tắt ứng dụng khách, họ sẽ nhập lệnh "exit" trên Cli. Điều này khiến Cli bắt đầu quy trình tắt Client, ứng dụng sẽ gửi một yêu cầu "stop" đến Tracker. Tracker sau đó xóa thông tin của Client khỏi Cơ sở dữ liệu và gửi một xác nhận về việc dọn dẹp thành công trở lại cho Ứng dụng khách. Ứng dụng khách xác nhận việc tắt với Cli, và Cli sẽ tự đóng lại, hoàn tất quá trình tắt máy.

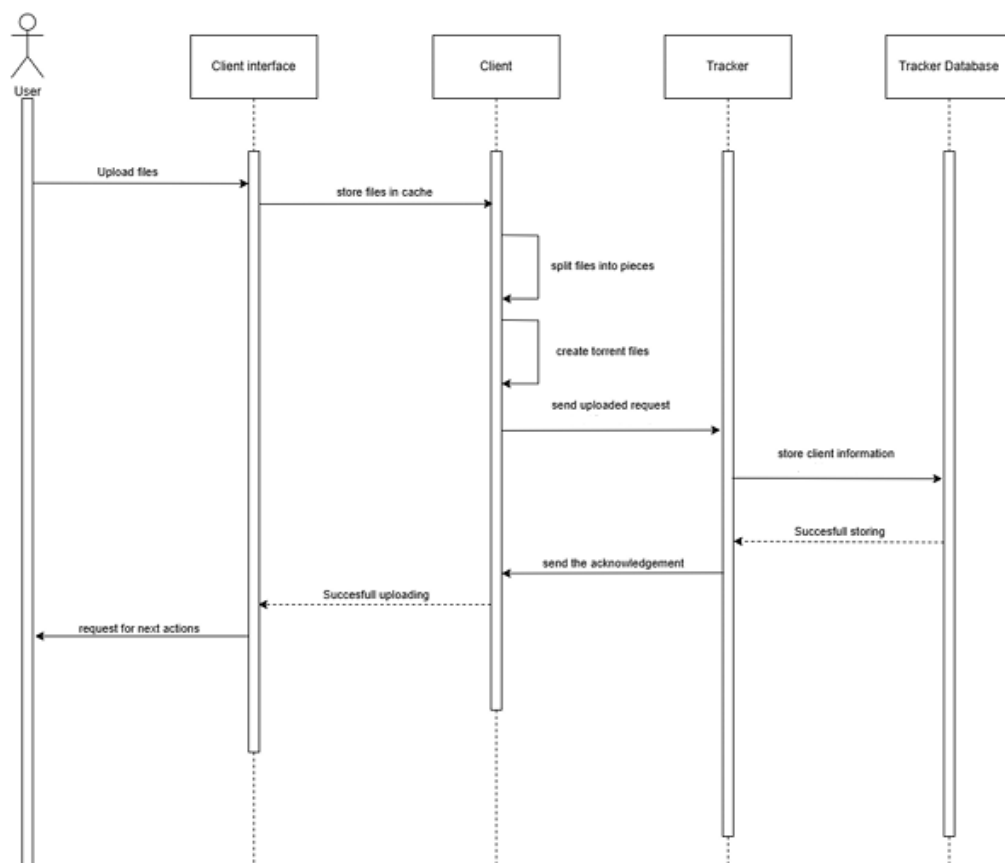


Hình 3: Sequence Diagram cho quá trình khởi động và tắt Client

6.2.2 File Uploading

Sequence Diagram (Hình ??) mô tả quy trình tải tệp lên với sự tương tác giữa người dùng, Client Cli, Client, Tracker và Tracker Database.

Quá trình bắt đầu khi người dùng chọn các tệp cần tải lên thông qua Client Cli. Cli này lưu trữ tệp vào bộ nhớ đệm tạm thời trước khi chuyển chúng đến Client. Tại đây, Client chia nhỏ các tệp thành nhiều phần, đồng thời tạo ra các tệp torrent để hỗ trợ chia sẻ P2P. Sau đó, Máy khách gửi yêu cầu "seed" đến Tracker. Tracker tiếp nhận, xử lý thông tin của Client và lưu trữ vào Tracker Database. Khi quá trình lưu trữ hoàn tất, Tracker gửi thông báo xác nhận ngược lại Client. Client tiếp tục thông báo trạng thái thành công cho Client Cli, từ đó hiển thị thông tin để người dùng biết tệp đã được tải lên và có thể thực hiện các thao tác tiếp theo. Toàn bộ quy trình đảm bảo việc quản lý dữ liệu từ khâu tiếp nhận, xử lý đến lưu trữ một cách chặt chẽ và minh bạch.

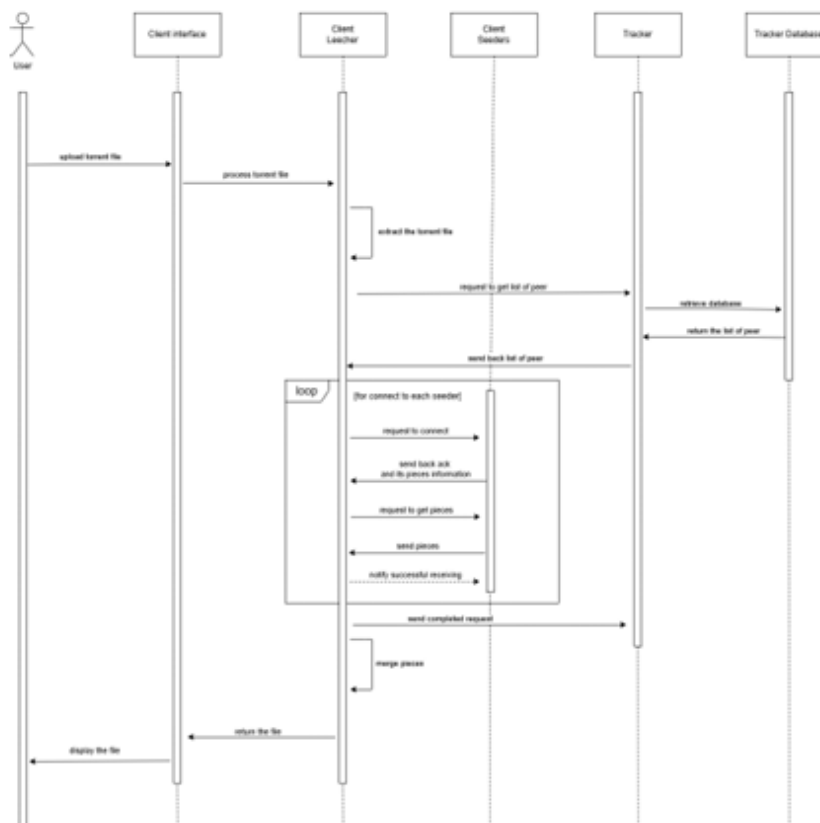


Hình 4: Sequence Diagram cho quá trình tải lên Client

6.2.3 File Downloading

Sequence Diagram (Hình 5) mô tả quy trình tải xuống tệp trong hệ thống dựa trên giao thức torrent, tập trung vào trường hợp người dùng (leecher) đã sở hữu sẵn tệp torrent.

Quá trình bắt đầu khi người dùng tải lên tệp torrent thông qua Client Cli. Cli này xử lý tệp và chuyển thông tin đến Client leecher. Tại đây, Client trích xuất dữ liệu từ tệp torrent và gửi yêu cầu đến tracker để nhận danh sách các peer (seeder) có khả năng cung cấp tệp. Tracker truy vấn cơ sở dữ liệu, lấy danh sách seeder và phản hồi lại cho Client leecher. Sau đó, Client leecher lần lượt kết nối với từng seeder trong danh sách, yêu cầu các phần tệp cụ thể. Mỗi seeder xác nhận yêu cầu, cung cấp thông tin về phần tệp mà họ lưu trữ, rồi gửi dữ liệu tương ứng đến Client leecher. Mỗi phần tệp nhận được sẽ được Client xác nhận thành công trước khi chuyển sang phần tiếp theo. Khi tất cả phần tệp được thu thập đầy đủ, Client leecher ghép chúng thành tệp hoàn chỉnh, gửi thông báo hoàn tất và trả kết quả về giao diện người dùng để hiển thị. Quá trình tương tác lặp đi lặp lại giữa leecher và các seeder này tiếp diễn cho đến khi việc tải xuống kết thúc, đảm bảo dữ liệu được truyền tải liên mạch và đầy đủ từ nhiều nguồn khác nhau.



Hình 5: Sequence Diagram cho quá trình tải xuống Client

7 Implementation and Testing

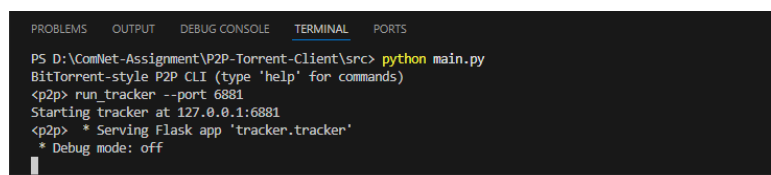
7.1 Implementation

Dự án của chúng tôi được triển khai bằng ngôn ngữ lập trình Python. Tracker - hoạt động như một máy chủ HTTP để xử lý các yêu cầu từ peer - được xây dựng dựa trên framework FastAPI. FastAPI là một framework web hiện đại, tốc độ cao (hiệu suất cao) dành cho việc phát triển API bằng Python, dựa trên gợi ý kiểu dữ liệu chuẩn của Python. Điều này giúp Tracker xử lý yêu cầu hiệu quả, đồng thời tận dụng ưu điểm về tốc độ và khả năng mở rộng của FastAPI để đáp ứng lượng lớn kết nối đồng thời từ các peer trong hệ thống P2P.

Source code: <https://github.com/tinta2510/torrent-like-application>. You can read the README to know about the details and manual documentation.

7.2 Testing

Do hạn chế về hạ tầng máy chủ, quá trình kiểm thử hệ thống được thực hiện trên môi trường localhost. Cả Tracker và Peer đều vận hành trên cùng địa chỉ IP 127.0.0.1 (localhost) nhưng sử dụng cổng (port) riêng biệt để phân biệt. Đầu tiên ta sẽ khởi động Tracker (Hình 6).

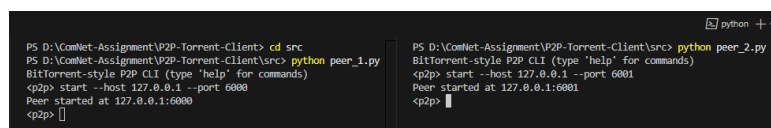


```
PS D:\ComNet-Assignment\VP2P-Torrent-Client\src> python main.py
BitTorrent-style P2P CLI (type 'help' for commands)
<p2p> run_tracker --port 6881
Starting tracker at 127.0.0.1:6881
<p2p> * Serving Flask app "tracker.tracker"
* Debug mode: off
```

Hình 6: Khởi động Tracker

Tiếp theo ta lần lượt khởi động Client1 (seeder) và Client2 (leecher) (Hình 7) bằng cách sử dụng lệnh `start --host [HOST] --port [PORT]`. Đây cũng chính là luồng xử lý chính để tiếp nhận các yêu cầu của người dùng bao gồm:

- **seed:** Bắt đầu seeding tập tin.
- **download:** Bắt đầu tải xuống tập tin.
- **status:** In ra thông tin về Peer.

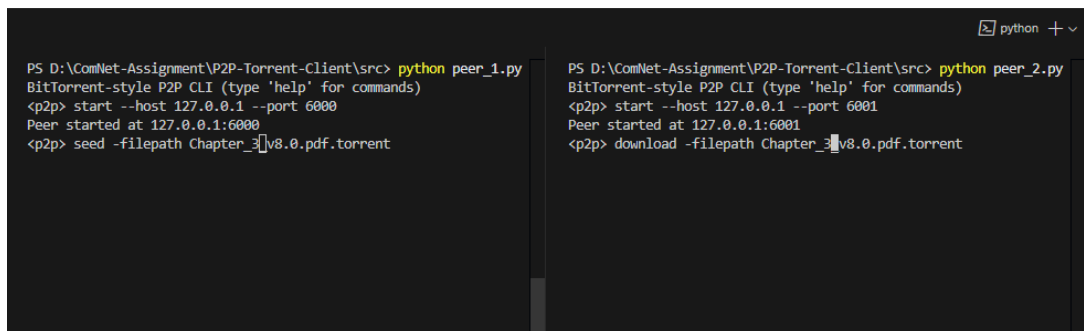


```
PS D:\ComNet-Assignment\VP2P-Torrent-Client> cd src
PS D:\ComNet-Assignment\VP2P-Torrent-Client\src> python peer_1.py
BitTorrent-style P2P CLI (type 'help' for commands)
<p2p> start --host 127.0.0.1 --port 6800
Peer started at 127.0.0.1:6800
<p2p>

PS D:\ComNet-Assignment\VP2P-Torrent-Client\src> python peer_2.py
BitTorrent-style P2P CLI (type 'help' for commands)
<p2p> start --host 127.0.0.1 --port 6801
Peer started at 127.0.0.1:6801
<p2p>
```

Hình 7: Khởi động Client

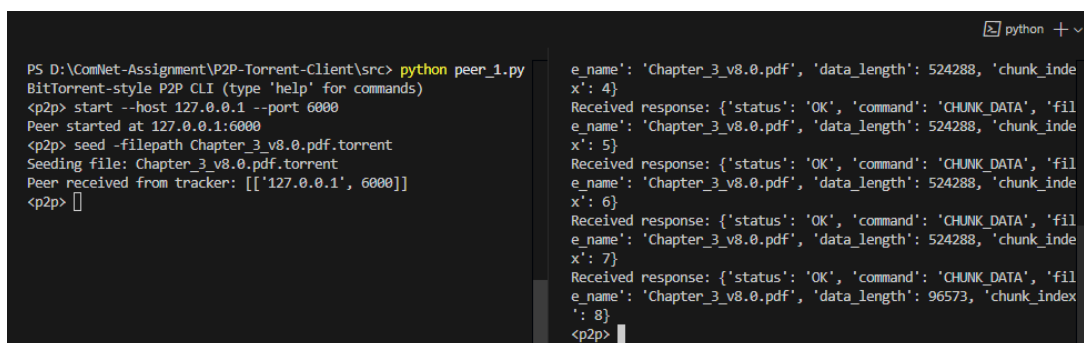
Sau khi khởi động xong, chúng ta lần lượt thực hiện lệnh **seed** cho Client1 và **download** cho Client2 để bắt đầu quá trình trao đổi tập tin (Hình 8). Trong quá trình trao đổi, các thông tin về tải xuống và logs sẽ được in trên Cli (Hình 9). Khi hoàn tất tải xuống, Client2 bây giờ sẽ trở thành seeder là tiếp tục quá trình chia sẻ (nếu có).



```
PS D:\ComNet-Assignment\P2P-Torrent-Client\src> python peer_1.py
BitTorrent-style P2P CLI (type 'help' for commands)
<p2p> start --host 127.0.0.1 --port 6000
Peer started at 127.0.0.1:6000
<p2p> seed -filepath Chapter_3_v8.0.pdf.torrent

PS D:\ComNet-Assignment\P2P-Torrent-Client\src> python peer_2.py
BitTorrent-style P2P CLI (type 'help' for commands)
<p2p> start --host 127.0.0.1 --port 6001
Peer started at 127.0.0.1:6001
<p2p> download -filepath Chapter_3_v8.0.pdf.torrent
```

Hình 8: Bắt đầu quá trình trao đổi file



```
PS D:\ComNet-Assignment\P2P-Torrent-Client\src> python peer_1.py
BitTorrent-style P2P CLI (type 'help' for commands)
<p2p> start --host 127.0.0.1 --port 6000
Peer started at 127.0.0.1:6000
<p2p> seed -filepath Chapter_3_v8.0.pdf.torrent
Seeding file: Chapter_3_v8.0.pdf.torrent
Peer received from tracker: [['127.0.0.1', 6000]]
<p2p>

e_name': 'Chapter_3_v8.0.pdf', 'data_length': 524288, 'chunk_index': 4}
Received response: {'status': 'OK', 'command': 'CHUNK_DATA', 'file_name': 'Chapter_3_v8.0.pdf', 'data_length': 524288, 'chunk_index': 5}
Received response: {'status': 'OK', 'command': 'CHUNK_DATA', 'file_name': 'Chapter_3_v8.0.pdf', 'data_length': 524288, 'chunk_index': 6}
Received response: {'status': 'OK', 'command': 'CHUNK_DATA', 'file_name': 'Chapter_3_v8.0.pdf', 'data_length': 524288, 'chunk_index': 7}
Received response: {'status': 'OK', 'command': 'CHUNK_DATA', 'file_name': 'Chapter_3_v8.0.pdf', 'data_length': 96573, 'chunk_index': 8}
<p2p>
```

Hình 9: Hoàn tất quá trình trao đổi file

8 Future Work

Dự án đã triển khai thành công các chức năng cốt lõi của hệ thống BitTorrent nhưng vẫn tồn tại hạn chế về khả năng vượt qua NAT (Cơ chế Dịch địa chỉ mạng). Điều này khiến các thiết bị nằm sau router NAT không thể kết nối trực tiếp với peer bên ngoài mạng cục bộ, làm giảm hiệu quả chia sẻ ngang hàng. Để khắc phục, phiên bản tương lai có thể tích hợp NAT-PMP và UPnP, giúp thiết bị sau NAT chia sẻ dữ liệu ra bên ngoài, nâng cao hiệu suất hệ thống. Ngoài ra, việc cho phép peer chia sẻ từng phần tệp đang tải xuống sẽ tăng số lượng seeder trong mạng lưới, thúc đẩy tốc độ và độ ổn định. Những cải tiến này hướng đến xây dựng hệ thống P2P linh hoạt, mạnh mẽ và dễ mở rộng trong mọi điều kiện mạng.

9 Tài liệu tham khảo

1. Slides bài giảng của thầy Nguyễn Lê Duy Lai.